Progress Report

Neural-Network-Based Time-Scale Modification Algorithm on Audio Signal

Shao-Hsuan Chu, Ju-Ting Chen
Instructor: Chia-Ping Chen

National Sun Yat-sen University

May 7, 2021

Abstract

We continue our model design from last time and read more carefully to get more details on the existing literatures. In this progress report, we will elaborate the effectiveness of the feature disentangle techniques. The instance normalization (IN) has been found effective on removing instance-specific-features while the adaptive instance normalization (AdaIN) can be used to inject the instance-specific-features into the final output. In addition, in order to improve the accuracy on the tempo estimation and enable end-to-end training, we decided to use neural networks to enhance the estimation of tempo, which is a one-step method to estimate tempo, unlike the conventional methods. Estimation methods often combine signal processing with heuristics. for example, used bandpass filters, followed by parallel comb filters, followed by peak picking. Several techniques on the neural-network-based tempo estimation have been proposed. We'll briefly discuss them in this report.

1 Introduction

In our previous report, we designed a model to separate the content and the tempo component with traditional tempo estimation (TE) algorithms. We expect the design can properly isolate the tempo information from others and thus achieving the time-scale modification (TSM) on the input audio. This time, we've switched from the traditional TE algorithms to a CNN-based TE

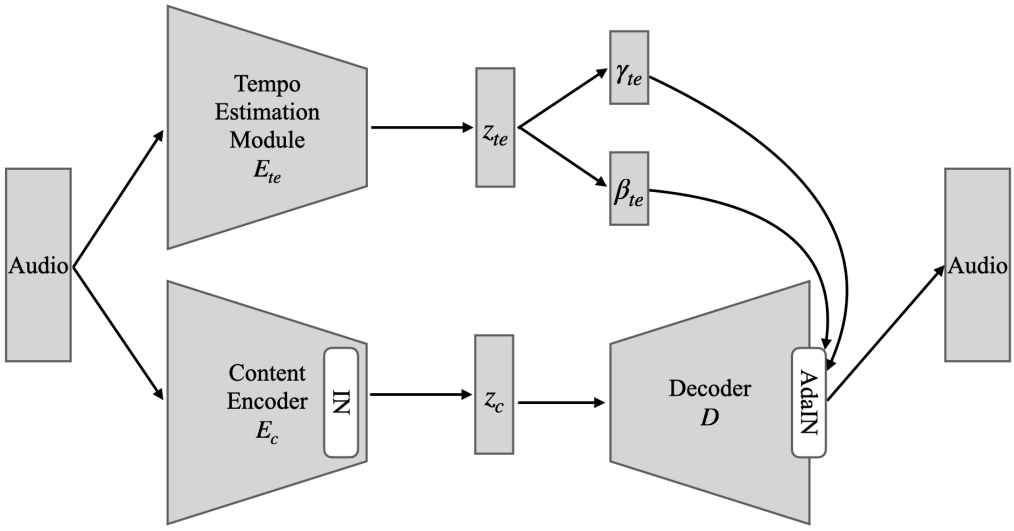

Figure 1: Model overview. E_c is the content encoder; E_{te} is the tempo estimation module and D is the decoder. IN is instance normalization layer without affine transformation. AdaIN represents adaptive instance normalization layer.

approach [3]. By which, we can now perform an end-to-end training and fine tune the TE model, producing more meaningful latent representation for the tempo information. See Figure 1 for the model architecture.

2 Feature disentangle

2.1 Method

2.1.1 Instance normalization

The instance normalization (IN) [5] is said to be capable of removing instance-specific-features, e.g., speaker information, image style, and hopefully tempo information [1][4]. The IN without affine transformation is given by

$$y = \frac{x - E[x]}{\sqrt{Var[x] + \epsilon}}$$

where the mean and standard-deviation are calculated per-channel separately for each sample in a mini-batch. That is to say, the mean and variance for each channel in each sample are equal to 0 and 1 respectively. From the

previous CNN research, we know that each convolution kernel represents certain kind of feature. In image, it could be texture, edge, style, or semantic meanings in the deep layers. In audio, it may be tones, tempo, speaker characteristic, etc. If we make each channel have the same mean and variance, they will lose the characteristic and be composed of mainly general informations that shared across the samples. Which hopefully, in our case, is everything except the tempo information.

2.1.2 Adaptive instance normalization

To further enhance the influence of the output from TE network. We use the same techniques of the enhancement on the speaker information in a feature-disentangle voice conversion model [1]. Chou et al. provide the speaker information to the decoder by adaptive instance normalization (adaIN) layer. In adaIN layer, the output from the decoder first got normalized by IN, then a channel-wise linear transformation, whose coefficients comes from the output of the TE network, will apply follow through. The formula is given by

$$y = \gamma_{te} \frac{D(x) - E[D(x)]}{\sqrt{Var[D(x)] + \epsilon}} + \beta_{te}$$

where γ_{te} and β_{te} are propagated from the TE latent vector z_{te} and the mean and standard-deviation are calculated per-channel separately for each sample in a mini-batch.

2.2 Evaluation of disentanglement

Chou et al. [1] perform some evaluation on their content encoder to see if the instance-specific-features are actually ripped out from it. They trained another fully connected network to classify speaker identity given the latent representation encoded by the content encoder. Three different configurations were placed:

1. Content encoder with IN
2. Content encoder without IN
3. Content encoder without IN while speaker encoder with IN

The results are shown in Table 1. The classification accuracy is apparently lower when IN is applied to the content encoder. However, the accuracy was not as high as expected for the content encoder without IN. This was probably because by the fact that the speaker encoder is able to control

E_c with IN	E_c without IN	E_c without IN and E_s with IN
0.375	0.658	0.746

Table 1: The accuracy for speaker identity prediction on content representation from Chou et al.

the channel statistics of decoder by adaIN, the whole model tends to learn speaker information from the speaker encoder rather than from the content encoder. The third setting is meant to verify this assumption, since we now apply IN on the speaker encoder, it can no longer hold as many speaker information as before and the highest accuracy on the evaluation network for this setting supports the notion.

3 Tempo estimation

3.1 Method

In the paper, Schreiber et al. describes a CNN-based approach that estimates the local tempo of a short musical piece or excerpt based on mel-scaled spectrograms in a single step [3]. without explicitly creating mid-level features like a beat activation function that need to be processed further by another, separate system component. And we can combine multiple local tempo into a global tempo by using averaging. It's a general system that can adapt to all kinds of data. In our training, we will use extended ballroom datasets, referred to as Eballs [2]. It contains 3,826 tracks with 30s length each.

3.2 Architecture

Although tempo estimation appears to be a regression problem, in the paper, the author approaches it as a classification problem. So instead of attempting to estimate a BPM value as decimal number, the model chooses one of 256 tempo classes, covering the integer tempo values from 30 to 285 BPM. First, process the input with three convolutional layers with 16 (1×5) filters for each. All filters are oriented along the time axis using padding and a stride of 1. These three layers are followed by four almost identical multi-filter modules (*mf_mod*, Figure 2, figure credit to Schreiber et al.), each consisting of an average pooling layer (m $\times$ 1), parallel convolutional layers with different filter lengths ranging from (1×32) to (1×256), a concatenation layer and a (1×1) bottleneck layer for dimensionality reduction. And add two fully connected layers (64 units each) followed by an output layer with 256 units

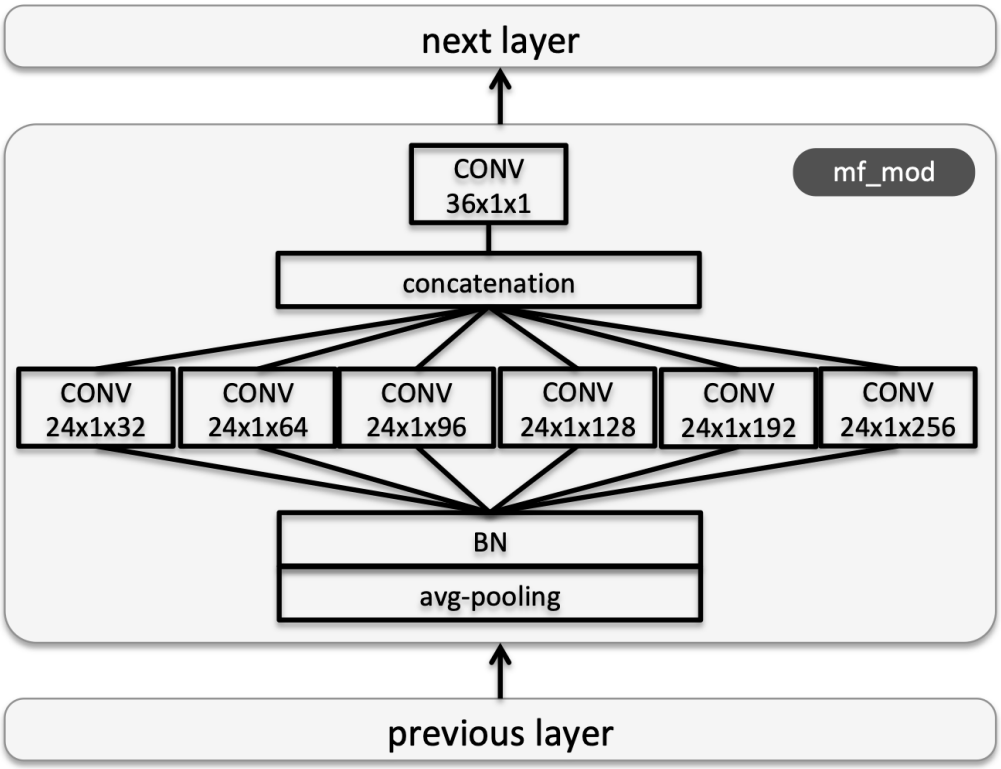

Figure 2: Each multi-filter module *mf_mod* consists of a pooling layer, batch normalization, six different convolutional layers, a concatenation layer and a bottleneck layer. The activation function for all convolutional layers is ELU.

to classify the features delivered by the convolutional layers. The first fully connected layer is additionally preceded by a dropout layer with p = 0.5 to counter overfitting. As loss function use categorical cross-entropy. Overall, the network has 2,921,042 trainable parameters.

References

[1] CHIEH CHOU, J., AND LEE, H.-Y. One-Shot Voice Conversion by Separating Speaker and Content Representations with Instance Normalization. In *Proc. Interspeech 2019* (2019), pp. 664–668.

[2] MARCHAND, U., AND PEETERS, G. The Extended Ballroom Dataset, Aug. 2016. Late-Breaking Demo Session of the 17th International Society for Music Information Retrieval Conf.. 2016.

[3] SCHREIBER, H., AND MÜLLER, M. A single-step approach to musical tempo estimation using a convolutional neural network.

[4] ULYANOV, D., VEDALDI, A., AND LEMPITSKY, V. Improved texture networks: Maximizing quality and diversity in feed-forward stylization and texture synthesis, 2017.

[5] ULYANOV, D., VEDALDI, A., AND LEMPITSKY, V. Instance normalization: The missing ingredient for fast stylization, 2017.